Now that we understand the concept of relevance and the controlling factors (precision and recall), the rest of the chapter is dedicated to working with the full-text queries. We begin by getting some sample data.

10.2 Sample data

In this chapter, we work with a fictional book store. We index a set of 50 technical books into an index named `books` by invoking the `_bulk` API. We do not tweak the mapping for this part of the sample data, so you can index the books as is. The data for the books is available on GitHub at http://mng.bz/zX2g, and the script to index the books is at http://mng.bz/0KzW; the files are also available from the book's website (https://www.manning.com/books/elasticsearch-in-action-second-edition).

Elasticsearch provides a number of full-text queries. Because each query type has many implementation details, they are described in separate sections for ease of flow. The first query we look at is the `match_all` query, which returns everything.

10.3 The match_all query

As the name suggests, the match all (`match_all`) query fetches all the documents in the index. Because this query is expected to return all available documents, it is the perfect partner for honoring 100% recall.

10.3.1 Building the match_all query

We form the query with a `match_all` object, passing it no parameters. The code in the following listing shows how to build a `match_all` query.

Listing 10.1 Fetching all documents using `match_all`

```
GET books/_search
{
  "query": {
    "match_all": { }
  }
}
```

The match_all query with no parameters

This query returns all documents available in the `books` index. The notable point is that the response indicates each book has a score of 1.0 by default:

```
{
 "max_score" : 1.0,
 ...
 "hits" : [{
   "_index" : "books",
   "_type" : "_doc",
   "_id" : "2",
   "_score" : 1.0,
   "_source" : {
     "title" : "Effective Java",
       ...
     }
```